«Util»

MyString

data : char*
len : int

length() : int
c_str() : const char*
toInt() : int
toFloat() : float
substring(start, count) : MyString
indexOf(c) : int

«Util»

MyVector

T

cap : int
arr : T*
current : int

add(element : T) : void
remove(index : int) : void
getSize() : int

«Util»
Timer

seconds : int
minutes : int
hours : int
paused : bool

startTimer() : void
pauseTimer() : void
updateTime() : void
resetTimer() : void

«Util»

Queue

buffer : char*
count : int

addChar(c : char*) : void
pop() : void
getCount() : int

«Util»
Animation

totalFrames : int
currentFrame : int
frameInterval : float
isComplete : bool
active : bool
scaleX : float
scaleY : float
useCustomFrames : bool

update() : void
reset() : void
loadTexture(path : const char*) : void
getTexture() : Texture&
isActive() : bool
getIsComplete() : bool
setScale(x, y : float) : void

«Util»

LoopAnimation

update() : void

«Util»
OneShotAnimation

update() : void

«Util»

SfmlObject

x : float
y : float
Tex : Texture

sprite : Sprite
width : float
height : float
scaleX : float
scaleY : float

centreSprite() : void
getX() : float
getY() : float
getWidth() : float
getHeight() : float
setPosition(x, y : float) : void
setScale(scaleX, scaleY : float) : void

«System»
DamageZone

x : float
y : float
width : float
height : float
damage : int
damagesPlayersFlag : bool
piercesShield : bool
consumed : bool

getDamage() : int
damagesPlayers() : bool
getPiercesShield() : bool
{virtual} igniteEntity(victim : DamagableEntity*) : void
{virtual} applySpecialEffect(player : PlayerSoldier*) : void
{virtual} update() : void
{virtual} onHit() : void
{virtual} shouldBeRemoved() : bool

«System»
OneShotZone

onHit() : void
shouldBeRemoved() : bool

«System»
TimedZone

remainingLife : float

update() : void
shouldBeRemoved() : bool

«System»
ExplosionZone

shouldBeRemoved() : bool

«System»
FireZone

shouldBeRemoved() : bool
igniteEntity(victim : DamagableEntity*) : void

«System»
ZombieHitZone

applySpecialEffect(player : PlayerSoldier*) : void

«System»
MummyCurseZone

applySpecialEffect(player : PlayerSoldier*) : void

«Entity»
Entity

entityId : int
velocityX : float
velocityY : float
max_speed : float
acceleration : float
onGround : bool

checkCollision(other : Entity*) : bool
calculateDistance(other : Entity*) : float
{virtual} movement() : void
{virtual} updateAnimations() : void
{virtual} behaviour() : void
{virtual} receiveDamageZone(zone : DamageZone&) : void
{virtual} shouldBeRemoved() : bool
{virtual} isOffScreen() : bool
{virtual} isFinalBoss() : bool
{virtual} inVehicle() : bool
{virtual} writeStateTo(s : CurrentState&) : void
{virtual} applyStateFrom(s : CurrentState&) : void
{virtual} setAndDraw() : void
{virtual} isHostileToPlayers() : bool
getId() : int

«Level»
Level

levelComplete : bool
screenLeft, screenRight : int
screenTop, screenBottom : int
timeTillRain : float
isRaining : bool

generateMap() : void
renderScreen() : void
{virtual} scrollLevel() : void
{virtual} checkCompletion(em : EntityManager*) : void
{virtual} hasFiniteBounds() : bool
{virtual} tileAtWorld(worldX, worldY : float) : char
addDamageZone(zone : DamageZone*) : void
queueSpawn(e : Entity*) : void
isLevelComplete() : bool
updateRain() : void
getDisplayName() : MyString

«Entity»
DamagableEntity

HP : int
isAttacking : bool
facingRight : bool
dead : bool

getHP() : int
getFacingRight() : bool
attack() : void
{virtual} onMeleeStart() : void
{virtual} onMeleeEnd() : void
{virtual} onIgnite() : void
{virtual} changeHP(change : int) : void
{virtual} absorbsHit(zone : DamageZone&) : bool
isDead() : bool
{virtual} getShootAngle() : float
receiveDamageZone(zone : DamageZone&) : void
shouldBeRemoved() : bool
setAndDraw() : void

«Entity»
Block

HP : int

behaviour() : void

«Level»

Map

lvl : char**
cell_size : int
height : int
width : int
chunkOriginX : long long

getLvl() : char**&
setTile(x, y : int, tile : char) : void
getCellSize() : int
getHeight() : int
getChunkOriginX() : long long

«Entity»
InteractableObject

interacted : bool

collect(owner : DamagableEntity*) : void

«Entity»
Collectible

collected : bool

isCollected() : bool
{virtual} collect() : void

«Weapon»
Weapon

damageDealt : float
range : float
angle : float
fireRate : float
cooldownTimer : float
baseFireRate : float
muzzleOffsetX : float
muzzleOffsetY : float

fire() : void
{virtual} update() : void
{virtual} reload() : void
{virtual} addAmmo(delta : int) : void
getDamage() : float
{virtual} isOutOfAmmo() : bool
{virtual} boostFireRate(multiplier : float) : void
{virtual} fireAtAngle(angle : float) : void
{virtual} scaleBlastRadius(multiplier : float) : void
{virtual} getDisplayName() : MyString
collect(owner : DamagableEntity*) : void

«Weapon»
ProjectileWeapon

totalAmmo : int
currentMag : int
magazineSize : int
reloadTime : float
reloadTimer : float
reloading : bool

fire() : void
{virtual} reload() : void
{virtual} hasInfiniteAmmo() : bool
addAmmo(amount : int) : void
getTotalAmmo() : int
isReloading() : bool
isOutOfAmmo() : bool

«Weapon»
Knife

pierce : bool
attacking : bool

fire() : void
canPierceShield() : bool
getDisplayName() : MyString

«Weapon»
Pistol

fire() : void
reload() : void
hasInfiniteAmmo() : bool
getDisplayName() : MyString

«Weapon»
HeavyMachineGun

fire() : void
getDisplayName() : MyString

«Weapon»
RocketLauncher

blastRadius : float

fire() : void
scaleBlastRadius(multiplier : float) : void
getDisplayName() : MyString

«Weapon»
FlameShot

fire() : void
getDisplayName() : MyString

«Weapon»
LaserGun

fire() : void
getDisplayName() : MyString

«Weapon»
PodLaserGun

fire() : void

«Weapon»
HandGrenade

throwOffset : float
blastRadius : float
ammo : int
throwSpeed : float

fire() : void
{virtual} onImpact() : void
addAmmo(amount : int) : void
scaleBlastRadius(multiplier : float) : void
isOutOfAmmo() : bool
getDisplayName() : MyString

«Weapon»
FireBombGrenade

fire() : void
onImpact() : void
getDisplayName() : MyString

«Entity»
Projectile

damagesPlayers : bool
damage : int
removeMe : bool

movement() : void

«Entity»
StraightProjectile

angle : float
distanceTraveled : float
range : float

getAngle() : float
{virtual} movement() : void

«Entity»
BallisticProjectile

{virtual} movement() : void

«Entity»
Bullet

behaviour() : void
shouldBeRemoved() : bool

«Entity»
Rocket

blastRadius : float

behaviour() : void
shouldBeRemoved() : bool

«Entity»
Laser

lifetime : int

behaviour() : void
shouldBeRemoved() : bool

«Entity»
Grenade

blastRadius : float

{virtual} onExplode() : void
behaviour() : void
shouldBeRemoved() : bool

«Entity»
FireBomb

onExplode() : void

«Entity»
Explosion

blastRadius : float
done : bool

behaviour() : void
shouldBeRemoved() : bool

«Entity»
Fire

lifetime : float
damagePerTick : int
damagesPlayers : bool

behaviour() : void
shouldBeRemoved() : bool

«Entity»
Food

getSaturationAmount() : int
behaviour() : void
shouldBeRemoved() : bool

«Entity»
Turkey

getSaturationAmount() : int

«Entity»
Fruit

getSaturationAmount() : int

«Entity»
DroppedWeapon

collect() : void
behaviour() : void
shouldBeRemoved() : bool

«Entity»
Prisonor

collect(owner : DamagableEntity*) : void
behaviour() : void

«Entity»
SupplyBox

grenadeCount : int
fireBombCount : int
hasFireBombs : bool
opened : bool

collect(owner : DamagableEntity*) : void
behaviour() : void
shouldBeRemoved() : bool

«System»
TransformationState

currentState : int
stateTimer : float

applyEffects(player : PlayerSoldier*) : void
update(player : PlayerSoldier*) : void
removeEffects(player : PlayerSoldier*) : void
getCurrentState() : int
getStateTimer() : float

«System»
BaseState

applyEffects(player : PlayerSoldier*) : void
update(player : PlayerSoldier*) : void
removeEffects(player : PlayerSoldier*) : void

«System»
UndeadState

applyEffects(player : PlayerSoldier*) : void
update(player : PlayerSoldier*) : void
removeEffects(player : PlayerSoldier*) : void

«System»
MummyState

applyEffects(player : PlayerSoldier*) : void
update(player : PlayerSoldier*) : void
removeEffects(player : PlayerSoldier*) : void

«Player»
Soldier

«Player»
Inventory

currentWeapon : int
melee : bool
mummyState : bool

«Player»
PlayerSoldier

lives : int
state : int
damageState : int
saturation : int
fat : bool
isJumping : bool
immortal : bool
isCrouching : bool
powerupActive : bool
powerupTimer : float
fusionCooldown : float

powerup() : void
getMaxHP() : int
{virtual} movement() : void
{virtual} attack(): void
{virtual} applyFireRate() : void
{virtual} slugBonus() : void
{virtual} vehicleFireRateMultiplier() : void
{virtual} vehicleDuribilityMultiplier() : void
{virtual} extraLife() : void
{virtual} movementMultiplier() : void
{virtual} blastRadiusMultiplier() : void
{virtual} ammoMultiplier() : void
{virtual} meleePierce() : void
{virtual} hasMelee() : bool
{virtual} powerupEnd() : void
{virtual} meleeDamageMultipler() : void
{virtual} switchTransformationState(newState : TransformationState*) : void
respawn() : void
getLives() : int
drawHUD() : void
updateAnimations() : void
getShootAngle() : float
behaviour() : void
setAndDraw() : void
isHostileToPlayers() : bool

«Vehicle»
Vehicle

movement() : void
updateAnimations() : void
attack() : void
behaviour() : void
getWeapon() : Weapon*

«Player»
Marco

meleePierce() : void
powerup() : void
applyFireRate() : void
attack() : void
getMaxHP() : int
powerupEnd() : void

«Player»
Tarma

vehicleFireRateMultiplier() : void
vehicleDuribilityMultiplier() : void
extraLife() : void
powerup() : void
powerupEnd() : void
movementMultiplier() : void
getMaxHP() : int

«Player»
Eri

fWasPressed : bool

blastRadiusMultiplier() : void
powerup() : void
applyFireRate() : void
hasMelee() : bool
attack() : void
getMaxHP() : int

«Player»
Germi

meleeDamageMultipler() : void
powerup() : void
applyFireRate() : void
powerupEnd() : void
attack() : void
getMaxHP() : int

«AI»

EnemyAIState

{virtual} movement(enemy : Enemy*) : void
{virtual} attack(enemy : Enemy*) : void

«AI»
RebelAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»
ShieldedAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»
BazookaAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»
GrenadeAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»

ParatrooperAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»

MummyAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»
ZombieAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»
PodAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«AI»
MartianGroundedAIState

movement(enemy : Enemy*) : void
attack(enemy : Enemy*) : void

«Enemy»
Enemy

aggression : float
alerted : bool
patrolDirection : float
detectionRadius : float
patrolPauseTimer : float

getBatchSize() : int
behaviour() : void
{virtual} movement() : void
{virtual} attack() : void
canBeKilledWhenFar() : bool
isHostileToPlayers() : bool

«Enemy»
RebelSoldier

dying : bool
killedByBlast : bool

getBatchSize() : int
behaviour() : void

«Enemy»
ShieldedSoldier

dying : bool
killedByBlast : bool

getBatchSize() : int
behaviour() : void
absorbsHit(zone : DamageZone&) : bool

«Enemy»
BazookaSoldier

dying : bool
killedByBlast : bool

getBatchSize() : int
behaviour() : void

«Enemy»
GrenadeSoldier

dying : bool
killedByBlast : bool

getBatchSize() : int
behaviour() : void

«Enemy»
ParaTrooper

getBatchSize() : int
behaviour() : void
isLanded() : bool

«Enemy»
MummyWarrior

isCrumbled : bool
resurrectionTimer : float
meleeCooldown : float
isDying : bool
killedByFire : bool

getBatchSize() : int
behaviour() : void
absorbsHit(zone : DamageZone&) : bool
onIgnite() : void
IsCrumbled() : bool

«Enemy»
Zombie

meleeCooldown : float
dying : bool
killedByBlast : bool

getBatchSize() : int
behaviour() : void

«Enemy»
Martian

phase : int
podHP : int
transitioning : bool
killedByBlast : bool
dying : bool

getPhase() : int
getBatchSize() : int
behaviour() : void
changeHP(change : int) : void

«AI»

MahoragaAIState

{virtual} movement(m : Mahoraga*) : void
{virtual} attack(m : Mahoraga*) : void

«Entity»
Mahoraga

lifeTimer : float

setNearestTarget(e : Entity*) : void
attack() : void
behaviour() : void
getShootAngle() : float
isHostileToPlayers() : bool

«Vehicle»

EnemyVehicle

alerted : bool
detectionRadius : float
dying : bool

getBatchSize() : int
changeHP(change : int) : void
isHostileToPlayers() : bool

«Vehicle»

FlyingTara

getBatchSize() : int
behaviour() : void

«Vehicle»

Bradley

getBatchSize() : int
behaviour() : void

«Vehicle»

EnemySub

getBatchSize() : int
behaviour() : void

«Vehicle»

PlayerVehicle

isOccupied : bool
currentWeapon : int

{virtual} enter(player : PlayerSoldier*) : void
{virtual} exit() : void
getIsOccupied() : bool
getOccupant() : PlayerSoldier*
getWeapon() : Weapon*
changeHP(change : int) : void
getShootAngle() : float
isHostileToPlayers() : bool

«Vehicle»
GroundVehicle

movement() : void
attack() : void

«Vehicle»
AerialVehicle

movement() : void
attack() : void

«Vehicle»

AquaticVehicle

movement() : void
attack() : void

«Vehicle»
VehicleMode

movement(host : PlayerVehicle*) : void
attack(host : PlayerVehicle*) : void
updateAnimations(host : PlayerVehicle*) : void
applySprite(host : PlayerVehicle*) : void
getCurrentWeapon() : Weapon*
swapWeapon(slot : int) : void

«Vehicle»
GroundForm

currentWeapon : int

movement(host : PlayerVehicle*) : void
attack(host : PlayerVehicle*) : void
getCurrentWeapon() : Weapon*
swapWeapon(slot : int) : void

«Vehicle»
AerialForm

currentWeapon : int

movement(host : PlayerVehicle*) : void
attack(host : PlayerVehicle*) : void
getCurrentWeapon() : Weapon*
swapWeapon(slot : int) : void

«Vehicle»

AquaticForm

currentWeapon : int

movement(host : PlayerVehicle*) : void
attack(host : PlayerVehicle*): void
getCurrentWeapon() : Weapon*
swapWeapon(slot : int) : void

«Vehicle»

MetalSlug

getBatchSize() : int
behaviour() : void
getWeapon() : Weapon*

«Vehicle»

SlugFlyer

getBatchSize() : int
behaviour() : void
getWeapon() : Weapon*

«Vehicle»

SlugMariner

getBatchSize() : int
behaviour() : void
getWeapon() : Weapon*

«Vehicle»

AmphibiousSlug

currentForm : int

getBatchSize() : int
behaviour() : void
getWeapon() : Weapon*

«Boss»
Boss

currentWeapon : int
batchCooldown : float
retreating : bool
maxHP : int
detectionRadius : float

attack() : void
movement() : void
updateAnimations() : void
{virtual} spawnNextBatch() : void
behaviour() : void
playerInRange() : bool
isRetreating() : bool
isHostileToPlayers() : bool
getShootAngle() : float

«Boss»
IronNokana

weaponRotateTimer : float

behaviour() : void
changeHP(change : int) : void

«Boss»
HairBuster

weaponRotateTimer : float

behaviour() : void
changeHP(change : int) : void

«Boss»
SeaSatan

weaponRotateTimer : float

behaviour() : void
changeHP(change : int) : void

«Boss»
BossMode

hp : int
minionTimer : float

attack(host : UltimateBoss*) : void
movement(host : UltimateBoss*) : void
updateAnimations(host : UltimateBoss*) : void
spawnMinionBatch(host : UltimateBoss*) : void
{virtual} tickMinions(host : UltimateBoss*) : void
applySprite(host : UltimateBoss*) : void
getHP() : int
changeHP(delta : int) : void
isDead() : bool

«Boss»
GroundMode

currentWeapon : int
weaponRotateTimer : float

attack(host : UltimateBoss*) : void
movement(host : UltimateBoss*) : void
spawnMinionBatch(host : UltimateBoss*) : void
applySprite(host : UltimateBoss*) : void

«Boss»
AerialMode

currentWeapon : int
weaponRotateTimer : float

attack(host : UltimateBoss*) : void
movement(host : UltimateBoss*) : void
spawnMinionBatch(host : UltimateBoss*) : void
applySprite(host : UltimateBoss*) : void

«Boss»

AquaticMode

currentWeapon : int
weaponRotateTimer : float

attack(host : UltimateBoss*) : void
movement(host : UltimateBoss*) : void
spawnMinionBatch(host : UltimateBoss*) : void
applySprite(host : UltimateBoss*) : void

«Boss»
UltimateBoss

currentModeIndex : int
modeTimer : float
modeDuration : float
transitioning : bool
introPlaying : bool

behaviour() : void
changeHP(delta : int) : void
shouldBeRemoved() : bool
isFinalBoss() : bool
getCurrentMode() : BossMode*

«Level»
SurvivalLevel

isBossLevel : bool
currentLevel : int

generateMap() : void
renderScreen() : void
checkCompletion(em : EntityManager*) : void
getDisplayName() : MyString

«Level»
InfiniteLevel

profileChoice : int
worldSeed : int

getHeight(x : long long) : int
generateMap() : void
generateChunk(anyXInChunk : long long) : Map*
renderScreen() : void
scrollLevel() : void
tileAtWorld(worldX, worldY : float) : char
hasFiniteBounds() : bool
getDisplayName() : MyString

«Level»
HashValue

grad : float
val : float

seed(x : long long, globalRandomConst : int) : void
getGrad() : float
getVal() : float

«Level»
PerlinGenerator

worldSeed : int

noiseAt(x : long long, spacing : int) : float

«Level»
NoiseProfile

octaves : int
baseSpacing : int
baseAmplitude : float
heightMultiplier : int
heightOffset : int

{virtual} shape(n : float) : float

«Level»

AmplifiedProfile

shape(n : float) : float

«Level»
FlatProfile

shape(n : float) : float

«Level»
NormalProfile

«Level»
FractalNoiseGenerator

worldSeed : int

generateSummedHeights(anyXInChunk : long long, outBuffer : int*, profile : NoiseProfile*) : void

«Level»

NoiseProfileFactory

selectProfile(selectedProfile : int) : void
getProfile() : NoiseProfile*

«System»
FileHandler

loadFile(name : const char*) : void
readFile(name : const char*) : void
writeFile(name : const char*) : void
appendFile(name : const char*) : void
saveFile(name : const char*) : void

«System»
ProfileFileHandler

loadFile(name : const char*) : void
readFile(name : const char*) : void
writeFile(name : const char*) : void
appendFile(name : const char*) : void
saveFile(name : const char*) : void

«System»
RageBaitFileHandler

loadFile(name : const char*) : void
readFile(name : const char*) : void
writeFile(name : const char*) : void
appendFile(name : const char*) : void
saveFile(name : const char*) : void

«System»
ForksOfFateFileHandler

open : bool

openTimeline(name : const char*) : void
closeTimeline() : void
appendSnapshot(s : CurrentState&) : void
readSnapshotAt(name : const char*, index : int, dst : CurrentState&) : bool
snapshotCount(name : const char*) : int

«System»
Parameter

id : char*
name : char*
value : float
weight : float
max : float
min : float

getValue() : float
setValue(v : float) : void
getId() : char*
getWeight() : float

«System»
Profile

count : const int

getById(id : const char*) : Parameter*
setValue(id : const char*, value : float) : void
getValue(id : const char*) : float

«System»
Recorder

frameCount : int

setTrackedPlayer(player : PlayerSoldier*) : void
update() : void

«System»
EntityManager

totalPlayers : int
totalEnemies : int
devMode : bool
externalRewindActive : bool

addEntity(entity : Entity*) : void
runEntities() : void
removeEntity(index : int) : void
getCurrentPlayer() : PlayerSoldier*
scanAndSpawnEnemies(level : Level*) : void
spawnPlayers(level : Level*) : void
killFarAwayEnemies(centerX, radiusPx : float) : void
processDamageZones() : void
sweepDead() : void
drawHUD() : void
enableDevMode() : void
allPlayersDead() : bool
finalBossDefeated() : bool
setRewindActive(on : bool) : void

«System»
GhostController

{virtual} update() : void

«System»
Autopilot

timer : float
maxSeconds : const float
active : bool

activate() : void
isActive() : bool
update() : void

«System»
SessionManager

sessionCount : int

onGameLaunch() : void
onGameQuit() : void
getProfile() : Profile*

«System»
NearbyEnemyInfo

enemyID : int
enemyType : int
distanceFromPlayer : float

«System»
DeathFingerPrint

playerType : int
x : float
y : float
weaponType : int
ammoLeft : int
wasInVehicle : bool
damageState : int
nearbyEnemyCount : int
killerEnemyType : int
timeSinceCheckPoint : int

recordDeath(currentState : GameState&, killerID : int) : void
compareToSelf(other : DeathFingerPrint&) : float
compareToCurrent(gamestate : GameState&) : float
getKillerType() : int

«System»
DeathCompositor

FPcount : int

addFingerPrint() : void
compareCurrentToHistory(gamestate : GameState&) : int
increaseAggression() : void
spawnKillerEnemies() : void

«System»
States

id : int
x : float
y : float

«System»
PlayerState

lives : int
activeWeaponIndex : int
activeWeaponAmmo : int
vehicleID : int
immortal : bool
dead : bool
HP : int

«System»
EnemyState

HP : int
dead : bool

«System»
BossState

HP : int
currentWeapon : int
dead : bool

«System»
VehicleState

HP : int
occupiedByPlayerID : int
dead : bool

«System»
BulletState

angle : float
active : bool

«System»
CurrentState

timestamp : float
playerCount : int
enemyCount : int
bossCount : int
vehicleCount : int
bulletCount : int
screenLeft, screenRight : int
screenTop, screenBottom : int

«System»
StatesBuffer

writeHead : int
filled : int

push(s : CurrentState&) : void
peekRelative(framesAgo : int) : const CurrentState&
available() : int

«System»
ForksOfFateManager

isReversing : bool
rewindCursor : int
runTimer : float

beginSession(timelineFile : const char*) : void
endSession() : void
takeSnapshot() : void
startRewind() : void
stepRewind() : void
stopRewind() : void
reversing() : bool

«Audio»
WavHeader

chunkId : char[4]
chunkSize : int
sampleRate : int
numChannels : short
bitsPerSample : short

load(file : ifstream&) : void

«Audio»

Complex

real : float
imaginary : float

getreal() : float
operator+(other : Complex) : Complex
operator*(other : Complex) : Complex

«Audio»

Spectrum

size : int
originalSize : int
sampleRate : int

applyFilter(filter : Filter&) : void
coupling() : Whisper
getSampleRate() : int
getSize() : int

«Audio»
Filter

size : int

lowPass(cutoff : float, sampleRate : int, s : int) : void
highPass(cutoff : float, sampleRate : int, s : int) : void
bandPass(lower, upper : float, sampleRate, s : int) : void
getWeights() : float*

«Audio»

Whisper

size : int

convertto16bit() : void
Normalization() : void
Reversal() : void
Splicing(starttime, endtime : float) : Whisper
decoupling() : Spectrum
load(file : ifstream&) : void
display() : void

«Audio»

SampleAudio

load(filename : const char*) : void
loadFromMic(data : short*, size : int, header : WavHeader) : void
applyFFT() : void
getSpectrum() : Spectrum&

«Audio»
AudioChecker

loadReference(filename : const char*) : void
captureMic() : void
cosineSimilarity() : float
isMatch() : bool

«Audio»
EchoChamber

isSubmerged : bool

setAudio(audio : Whisper*) : void
update(playerInWater : bool) : void
getActiveAudio() : Whisper*
getIsSubmerged() : bool

«Audio»

AudioManager

getEchoChamber() : EchoChamber&
setAudio(audio : Whisper*) : void
match(filename : const char*) : bool

«System»
RageBait

runCount : int
currentMidScore : int
midPointPassed : bool

onMidPoint(currentScore : int) : void
onRunEnd(finalScore : int) : void

«Hardware»

MemoryModule

powerDraw : float

loadRawBinary(index : int, segment : unsigned short) : void
read(index : int) : unsigned char
write(index : int, value : unsigned char) : void

«Hardware»

RegisterBank

Flags : unsigned char

read(index : int) : unsigned short
write(index : int, value : unsigned short) : void

«Hardware»
Cache

block : unsigned char[16]
start : unsigned short
valid : bool

«Hardware»

SignalVector

IS_VALID : bool
USES_AEC : bool
AEC_OPERATION : int
IS_MEMORY_READ : bool
IS_MEMORY_WRITE : bool
IS_BRANCH : bool
WRITES_TO_REGISTER : bool

«Hardware»
Processor

plugged : bool
temperature : float
immediate16 : unsigned short

End() : bool
fetch() : void
decode() : int
execute() : void
plugIn(ab, db, cb, mb) : void
ALU(opcode, operandA, operandB, destReg) : int
getTemperature() : float

«Hardware»

PhosphorDisplay

row : int
col : int

insertChar(c : char) : void
render() : void

«Hardware»

GraphicsAdapter

powerDraw : float

CHAR_PAYLOAD(byte : unsigned char) : char
getPowerDraw() : float

«Hardware»

MyKeyboard

PENDING_INPUT() : bool
READ_CHAR() : int
READ_INT() : int

«Hardware»
PSU

maxWattage : float

checkPower(currentDraw : float) : void

«Hardware»
Mainboard

addressBus : unsigned short
dataBus : unsigned char
controlBus : unsigned char
powerOn : bool

READ(address : int) : void
WRITE(address, data : int) : void
pulseClock() : void
killPower() : void
isPowerOn() : bool
renderDisplay() : void

«System»
UrduTerminal

active : bool

RenderTerminal() : void
getActive() : bool
Activate() : void
Deactivate() : void

«AI»
NodeGene

id : int
type : int
value : float

getValue() : float
setValue(v : float) : void
sigmoid(x : float) : float

«AI»
ConnectionBase

fromNode : int
toNode : int
innovationNumber : int

getfromNode() : int
getToNode() : int
getInnovationNumber() : int

«AI»
ConnectionGene

weight : float
isEnabled : bool

getFromNode() : int
getToNode() : int
getWeight() : float
getIsEnabled() : bool

«AI»
Genome

nodeCount : int
connectionsCount : int
fitness : float

addNode(registry : InnovationRegistry&) : void
addConnection(registry : InnovationRegistry&) : void
tuneWeights() : void
crossOver(other : Genome&) : Genome
returnCompatibilityDistance(other : Genome&) : float
getFitness() : float

«AI»

InnovationRegistry

count : int

getInnovationCount(from, to : int) : int
reset() : void

«AI»

Species

genomeCount : int
bestFitness : float
generationsWithNoImprovement : int

isCompatible(other : Genome&) : bool
adjustFitness() : void
reproduce() : void

«AI»

Population

speciesCount : int
genomeCount : int
currentGeneration : int

speciate() : void
evaluate() : void
reproduce() : void
generationLoop() : void

«AI»
NeuralNetwork

takeInputs(inputs : float*, count : int) : void
propagate() : void
getOutput() : float*

«AI»

NEATtrackerHelper

currentIndex : int
totalGenomes : int
episodeTime : float
maxEpisodeTime : float
episodeRunning : bool

hasNext() : bool
getCurrentIndex() : int
startEpisode() : void
endEpisode() : void
shouldTerminate() : bool

«AI»
NEATController

runGeneration() : void
saveBestGenome() : void
loadBestGenome() : void
activateBestPlayer() : void

«System»
StateTable

type : int
totalTransitions : int
Table : int[7]
scoreGainedInNextState : int[7]
scoreGainedInThisState : int
timeSpendInThisState : float
Active : bool

ActivateState() : void
DeactivateState() : void
changeState(type : int) : void
isActive() : bool
highestProbability() : int
bestChain() : int

«System»
PressingState

«System»
ConsolidatingState

«System»
RetreatingState

«System»
BurningResourcesState

«System»
StallingState

«System»
SavingState

«System»
AFKState

«System»
StateManager

ActiveStateIndex : int

updateStates() : void
ActivateState() : void
manipulate() : void

«System»
GrudgefulSystem

update(screenLeft : int, playerX, playerY : float) : void

«System»
CowardiceSystem

reserveCount : int

update(screenLeft : int, playerX, playerY : float) : void

«System»
LevelManager

setLevel(level : Level*) : void
getCurrentLevel() : Level*
generateLevel() : void
RenderLevel() : void

«System»
GameManager

getFingerprintRecorder() : Recorder&
getAutopilot() : Autopilot&
getForksOfFateManager() : ForksOfFateManager&
getDeathCompositor() : DeathCompositor&
getUrduTerminal() : UrduTerminal&
getAudioManager() : AudioManager&
getGrudgefulSystem() : GrudgefulSystem&
getCowardiceSystem() : CowardiceSystem&
getRageBait() : RageBait&
getStateManager() : StateManager&

«System»
GameState

modeComplete : bool
developerMode : bool
currentGameMode : int

runGame() : void
activateDeveloperMode() : void
setModeComplete() : void
getModeComplete() : bool

«System»
PlayState

rWasPressed : bool

{virtual} runGame() : void

«System»
Survival

currentLevel : int

runGame() : void

«System»
Campaign

profileSelected : bool
chosenProfile : int

runGame() : void
selectProfileScreen() : void
buildLevel() : void

«System»
SelfPlay

bestPlayerActive : bool

toggleBestPlayer() : void
runGame() : void

«System»
MainMenu

selectedOption : int
detectedPopup : bool

displayMenu() : void
selectOption(option : int) : void
returnSelectedOption() : int

«System»
GameStateManager

currentGameMode : int

setGameState(gameMode : int) : void
run() : void

«System»
Game

run() : void

pendingSpawns

fireAnim

owner

owner

weapons
handGrenade

fireGrenade

vehicle

weapon

landingEnemy nearestTarget

weapon

occupant

weapons

weapons

weapons

weapons

modes[3]

currentBatch

weapon[3]

rocket

flame

firebomb

rocket

firebomb

hmg

rocket

modes[3]

introAnims

damageZones

currentPlayer

backgrounds

trackedPlayer

enemies

controlledPlayer
actualPlayer

enemies

nearbyEnemies[10]

players[4]

enemies[100]

bosses[4]

vehicles[10] bullets[200]

ring[600]

frequencyBins

test reference

muffledAudiooriginalAudio

enemyManager

signals[256]

data

connections nodes

globalInnovations

similarGenomes tester

globallyBest
source

currentPlayer

activeLevel

previousState

playerStates[7]

enemyManager

enemyManager

entities

pendingSpawns

damageZones

damageZones

pendingSpawns

profileMenu

popup menu